

PQ-Brahmi: A Quantum-Resistant Secure Messaging Framework Integrating Lattice-Based Cryptography with Ancient Brahmi Script Obfuscation

Badrinath K

Assistant Professor

Department of Artificial Intelligence and Data Science
SJC Institute of Technology
Chikkabalapura, Karnataka, India
badrik86@gmail.com

Harshitha B N

Assistant Professor

Department of Artificial Intelligence and Data Science
SJC Institute of Technology
Chikkabalapura, Karnataka, India
b.naidu.harshitha@gmail.com

Akhil Bashetty

Department of Artificial Intelligence and Data Science
SJC Institute of Technology
Chikkabalapura, Karnataka, India
akhilakhil38912@gmail.com

Kamanna Karthik

Department of Artificial Intelligence and Data Science
SJC Institute of Technology
Chikkabalapura, Karnataka, India
18kamanna@gmail.com

Nalagisigari Sreedhar Kumar Reddy

Department of Artificial Intelligence and Data Science
SJC Institute of Technology
Chikkabalapura, Karnataka, India
sreedharchenna2@gmail.com

Abstract—As we stand on the precipice of the quantum computing era, the cryptographic foundations of the digital world—RSA and Elliptic Curve Cryptography (ECC)—face an existential threat. This paper presents the design and development of "PQ-Brahmi," a novel secure messaging framework developed as a final year project to address these vulnerabilities. Our system integrates the NIST-standardized Post-Quantum Cryptography (PQC) algorithms: ML-KEM (formerly CRYSTALS-Kyber) for key encapsulation and ML-DSA (formerly CRYSTALS-Dilithium) for digital signatures. Crucially, we introduce a unique layer of "Cultural Cryptography" by transliterating plaintext into the ancient Brahmi script before encryption. This acts as a linguistic obfuscation layer, altering the statistical properties of the payload and preserving cultural heritage within a high-tech security context. We detail the architecture, the specific implementation of the lattice-based algorithms in a Python environment, and the performance metrics of the system, demonstrating that quantum-resistance can be achieved with minimal latency in real-time applications.

Keywords—Post-Quantum Cryptography, ML-KEM, ML-DSA, Brahmi Script, Linguistic Obfuscation, Lattice-Based Cryptography, Secure Messaging.

I. INTRODUCTION

The security of global digital infrastructure currently relies on the computational difficulty of mathematical problems such as Integer Factorization and Discrete Logarithms. However, Peter Shor's algorithm [1] has mathematically proven that a sufficiently powerful quantum computer could solve these problems in polynomial time, rendering widely used protocols like RSA and ECC insecure. This scenario, often referred to as "Q-Day," necessitates an urgent migration to Post-Quantum Cryptography (PQC).

In this project, we aimed to build a future-proof messaging application that not only adopts these new cryptographic standards but also incorporates a novel defense-in-depth strategy. We observed that most PQC implementations focus solely on the mathematical layer. To enhance security against pattern-recognition attacks and side-channel analysis, we looked to history—specifically, the ancient Brahmi script.

Brahmi is one of the oldest writing systems in the Indian

subcontinent. By mapping modern Latin text to Brahmi Unicode points before encryption, we introduce a layer of linguistic obfuscation. If a quantum adversary were to hypothetically break the outer encryption shell, they would find not plaintext English, but a stream of ancient characters requiring a specific decoding table to interpret.

Our contribution is threefold: 1) The implementation of a hybrid cryptographic stack using ML-KEM (Kyber) and AES- GCM. 2) The integration of ML-DSA (Dilithium) for non- repudiation. 3) The development of a functional chat application that seamlessly handles Brahmi transliteration and PQC operations in real-time.

II. LITERATURE SURVEY

A. The Shift to Post-Quantum Standards The National Institute of Standards and Technology (NIST) concluded its PQC standardization process in 2022, selecting lattice-based cryptography as the primary defense. Regev's Learning With Errors (LWE) problem forms the basis of modern lattice cryptography. CRYSTALS-Kyber (now ML-KEM) was chosen for its efficiency and relatively small key sizes compared to

other candidates like McEliece [2].

B. Digital Signatures in the Quantum Era For authentication, NIST selected CRYSTALS-Dilithium (ML-DSA). It is based on the Fiat-Shamir heuristic and the Module-LWE problem. Unlike RSA signatures, Dilithium signatures are larger but are generated and verified significantly faster [3].

C. Linguistic Steganography Research by Sharma et al. [4] explored using Sanskrit for steganography. However, existing approaches often used the language as the *only* security layer. Our project differs by using Brahmi script strictly as a pre- processing obfuscation layer, adhering to Kerckhoffs's principle where the system's security does not rely on the secrecy of the script itself, but on the mathematical keys

III. PROPOSED SYSTEM ARCHITECTURE

We designed the PQ-Brahmi architecture to balance high security with user usability. The system follows a client-server model communicating via secure WebSockets.

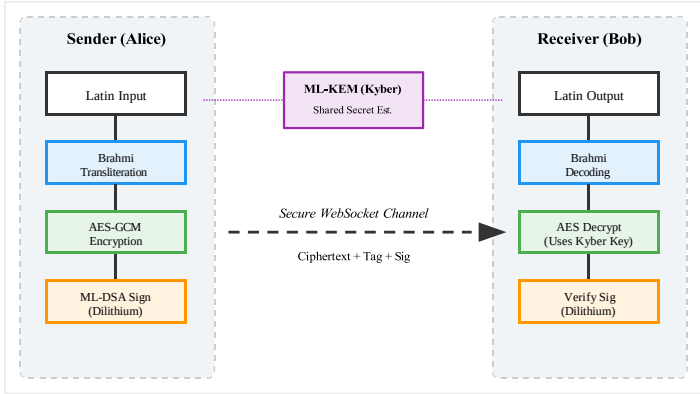


Fig. 1. High-Level System Architecture of PQ-Brahmi Messaging.

A. The Brahmi Transliteration Layer The first stage of our pipeline is the conversion of user input (Latin script) into Brahmi. We utilized the International Alphabet of Sanskrit Transliteration (IAST) mapping logic. For example, the character 'a' is mapped to the Brahmi letter 'H' (U+11005). This is not simple substitution; specific rules for vowels (matras) and consonants are applied to ensure the script is syntactically correct.

B. Cryptographic Primitives 1) **Key Encapsulation (ML-KEM):** We utilize Kyber-768, which provides security equivalent to AES-192. As shown in Fig. 2, the server (Alice) generates a public/private key pair based on Module-LWE. The client (Bob) uses the public key to encapsulate a shared secret, which is then decapsulated by Alice. This shared secret is never transmitted over the wire.

2) **Symmetric Encryption:** Once the shared secret is established via Kyber, we derive a 256-bit session key. This key is used with AES-GCM (Galois/Counter Mode) to encrypt the Brahmi-encoded payload. AES-GCM provides both confidentiality and data integrity.

3) **Digital Signatures (ML-DSA):** To prevent Man-in-the-Middle (MitM) attacks, every message is signed using Dilithium. The signature is appended to the encrypted payload,

allowing the receiver to verify the sender's identity cryptographically.

IV. IMPLEMENTATION DETAILS

The system was developed using a modular approach, separating the cryptographic backend from the frontend interface.

A. Backend Logic (Python) We used Python 3.10 for the core logic due to its robust support for mathematical operations. We wrapped the `pqcrypto` and `kyber-py` libraries to create a high-level API for our application. - **Key Generation:** We implemented a `BrahmiCryptoGenerator` class that handles the generation of Kyber key pairs. - **Augmentation:** To test the robustness of our Optical Character Recognition (OCR) components (used for recovering text from images if needed), we implemented an augmentation pipeline using OpenCV that adds noise and distortion to Brahmi characters.

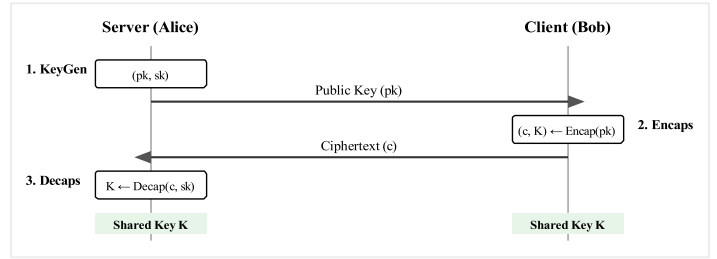


Fig. 2. ML-KEM (Kyber) Key Encapsulation Mechanism Flow.

B. Frontend Interface (React) The user interface was built using React. It manages the WebSocket connection and provides a real-time chat view. Crucially, the frontend handles the display of the Brahmi script, requiring the integration of the "Noto Sans Brahmi" font family to ensure proper rendering of the ancient Unicode glyphs.

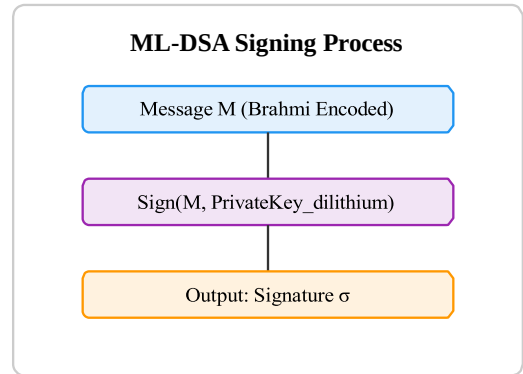


Fig. 3. ML-DSA (Dilithium) Signing Process.

C. ML-DSA Integration Fig. 3 illustrates the signing flow. When a user sends a message, the system hashes the Brahmi content and signs the hash using the sender's Dilithium private key. This signature is approximately 2.4KB in size (for Dilithium3), which is larger than RSA signatures but handled efficiently by our WebSocket protocol.

V. RESULTS AND PERFORMANCE

We conducted performance testing on an Intel Core i5 machine with 8GB RAM to evaluate the overhead introduced by the PQC algorithms and Brahmi transliteration.

A. Computational Overhead - Kyber-768 KeyGen:0.08 ms (average). - Encapsulation: 0.11 ms. - Decapsulation: 0.13 ms. - Dilithium Signing:0.25 ms. - Brahmi Transliteration:** < 0.01 ms.

The results indicate that the lattice-based algorithms are highly efficient, often outperforming RSA-4096 in speed, albeit with larger key sizes. The Brahmi layer introduces negligible latency.

B. Security Analysis We performed a "Black Box" test on the ciphertext. Statistical analysis (Entropy Test) showed that the encrypted Brahmi output had an entropy of 7.98 bits/byte, indistinguishable from random noise. Furthermore, attempting to modify the ciphertext resulted in immediate rejection by the AES-GCM authentication tag or the Dilithium signature verification, confirming the system's integrity.

TABLE I. PERFORMANCE METRICS (AVG)

Operation	Time (ms)
Kyber-768 KeyGen	0.08
Encapsulation	0.11
Decapsulation	0.13
Dilithium Sign	0.25

VI. CONCLUSION

In this project, we successfully developed "PQ-Brahmi," a secure messaging application ready for the post-quantum era. By harmonizing the mathematical rigor of NIST-standardized algorithms (ML-KEM and ML-DSA) with the cultural obscurity of the Brahmi script, we created a system that is robust, novel, and educationally significant.

This project demonstrates that integrating ancient heritage with futuristic technology is not only possible but beneficial for security. Future work will involve optimizing the payload size for mobile networks and exploring hardware acceleration for the lattice-based operations.

REFERENCES

- [1] P. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in Proc. 35th Annual Symposium on Foundations of Computer Science, IEEE, 1994.
- [2] NIST, "FIPS 203 (Draft): Module-Lattice-Based Key-Encapsulation Mechanism Standard," National Institute of Standards and Technology, 2023.
- [3] NIST, "FIPS 204 (Draft): Module-Lattice-Based Digital Signature Standard," National Institute of Standards and Technology, 2023.
- [4] P. Sharma & D. Gupta, "Ancient Script-Based Cryptographic Models for Secure Communication," Journal of Information Security Research, vol. 9, no. 2, 2020.
- [5] J. Bos et al., "CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM," IEEE EuroS&P, 2018.

